

# Architecture of a Production RAG Retrieval Pipeline

## Introduction

A retrieval-augmented generation pipeline is only as good as the weakest stage in the sequence of steps that turns a user's question into a final set of ranked chunks, and understanding where time and quality are gained or lost at each stage is necessary for reasoning about the system as a whole rather than optimizing any single component in isolation.

This document describes the end-to-end shape of the pipeline used in this system: optional query decomposition, embedding, vector search against PostgreSQL and pgvector, candidate pool assembly with deduplication, cross-encoder reranking, and the selection of a final top-K result set, along with how latency is budgeted across these stages and how the pipeline behaves when one of its stages is slow, unavailable, or returns an unexpected result.

## Stage-by-Stage Overview

A user query first passes through an optional decomposition stage, where a language model may split a single complex query into several retrieval-oriented subqueries while always retaining the original query text unchanged; this stage is optional in the sense that a simple query may pass through unmodified as a single subquery equal to the original, but the architectural step itself is always present in the request path.

Every resulting query, whether the original alone or the original plus generated subqueries, is independently embedded using the production embedding model, and each resulting vector is used to perform a separate HNSW-indexed cosine-distance vector search against the stored chunk embeddings in PostgreSQL, producing one ranked candidate list per query or subquery.

These independent candidate lists are then merged into a single candidate pool, deduplicated by chunk identifier so that a chunk retrieved by more than one subquery appears only once, with the lowest observed vector distance retained for any chunk that appeared under multiple subqueries, since a lower distance under any one formulation is treated as the chunk's best available relevance signal regardless of which specific subquery surfaced it.

The merged, deduplicated candidate pool, bounded by the configured ``candidate_limit``, is passed to a cross-encoder reranker together with the original user query, never the generated subqueries, which rescales every candidate's relevance score using a joint encoding of the query and that specific candidate's content, and the pool is reordered according to this new score before a final top-K slice, bounded by the configured ``final_limit``, is returned as the pipeline's output.

## Candidate Pool Sizing

The ``candidate_limit`` parameter controls how many chunks are carried forward out of vector search and into the merge-and-rerank stages, and it functions as a ceiling on how much raw recall the rest of the pipeline is even given the opportunity to work with, since a chunk that vector search does not place inside the top ``candidate_limit`` results can never be recovered by reranking, regardless of how relevant that chunk actually is.

An internal sweep across candidate pool sizes, holding the final result size fixed, found that recall measured against a fixed evaluation set continues to improve as ``candidate_limit`` increases, but with diminishing returns past a certain point, while end-to-end latency increases roughly in proportion to pool size because the reranker must score every candidate in the pool individually:

| candidate_limit | Mean Recall@10 before reranking | Mean cross-encoder rerank latency |
|-----------------|---------------------------------|-----------------------------------|
| 20              | 0.47                            | 32 ms                             |
| 50              | 0.61                            | 78 ms                             |
| 100             | 0.69                            | 145 ms                            |
| 200             | 0.74                            | 290 ms                            |

The diminishing-returns pattern in this table reflects that most queries have their true relevant chunks concentrated near the top of the vector-search ranking already, so a larger candidate pool mostly adds marginal, lower-ranked candidates that occasionally recover an additional relevant chunk but increasingly often simply adds more irrelevant material for the reranker to process without finding anything new. Because reranking latency scales with candidate pool size while recall gains taper off, `candidate\_limit` is fundamentally a latency-versus-recall dial rather than a parameter with one objectively correct setting, and the appropriate value depends on how much end-to-end latency a given deployment is willing to accept in exchange for the marginal recall improvement a larger pool provides.

## Final Limit Selection

The `final_limit` parameter determines how many reranked chunks are actually returned from the pipeline, and it interacts with precision in a straightforward way: a smaller final limit returns fewer chunks, so each one needs to be correct for precision to stay high, while a larger final limit tolerates some lower-relevance chunks appearing in the result set in exchange for a higher chance that all genuinely relevant chunks were included somewhere in the returned set.

Choosing `final_limit` independently of `candidate_limit` is a deliberate design decision: a large candidate pool can still be reranked down to a small final result set, which lets the pipeline cast a wide net for recall purposes during vector search while still returning a tightly curated final set of chunks, since reranking, not the final slice itself, is responsible for deciding which of the many candidates are worth surfacing.

A `final_limit` set larger than what the downstream consumer of the retrieved chunks can actually make good use of provides no additional benefit and only risks including lower-quality chunks near the bottom of the returned set, since reranking does not raise a candidate's absolute quality, it only orders the candidate pool that vector search already assembled.

## Merge and Deduplication Logic

When query decomposition produces more than one subquery, each subquery's vector search results are kept as a separate ranked list until the merge step, at which point every candidate across every subquery's result list is grouped by chunk identifier, and any chunk identifier that appears in more than one subquery's results is collapsed into a single entry retaining the lowest vector distance observed for that chunk across all the subqueries that surfaced it.

This lowest-distance-wins merge rule means a chunk that one subquery's vector search ranked only moderately well can still end up near the top of the merged candidate pool if a different subquery's vector search ranked that same chunk very strongly, which is specifically how decomposition is intended to recover evidence relevant to one part of a multi-part question even when that evidence would have ranked poorly against the original, undecomposed query taken as a whole.

Deduplication by chunk identifier happens strictly before reranking, not after, so the cross-encoder reranker never scores the same chunk twice under two different subqueries; it always sees exactly one entry per unique chunk in the merged pool, scored once against the single original query, regardless of how many subqueries originally surfaced that chunk during vector search.

## Latency Budget Across Stages

Single-query latency, meaning a query that does not trigger decomposition into multiple subqueries, is dominated by the cross-encoder reranking stage rather than by embedding or vector search, which is consistent with reranking being the only stage that must individually examine the full content of every candidate rather than operating on compact vector representations:

| Stage                                     | p50 latency | p95 latency | Notes                         |
|-------------------------------------------|-------------|-------------|-------------------------------|
| Query embedding                           | 6 ms        | 11 ms       | single query, MiniLM encode   |
| Vector search (HNSW, candidate_limit=100) | 18 ms       | 42 ms       | cosine distance operator      |
| Candidate merge and deduplication         | <1 ms       | 2 ms        | in-process, negligible cost   |
| Cross-encoder reranking (100 candidates)  | 140 ms      | 260 ms      | dominant cost in the pipeline |
| Total end-to-end (no decomposition)       | 165 ms      | 310 ms      | single original query only    |

When query decomposition is triggered, the dominant additional cost is not the extra embedding or vector search work from the additional subqueries, both of which remain individually fast and can be issued without

waiting on each other sequentially, but the network round trip to the OpenRouter-backed language model generating the subqueries in the first place, which typically adds on the order of 180 to 400 milliseconds depending on model load and subquery count, making decomposition's latency cost almost entirely attributable to one external call rather than to the additional retrieval work it causes downstream.

## Caching Strategies

Caching a query's embedding for repeated or near-identical queries avoids redundant encoding work on the fast embedding stage, which offers a small but real latency benefit, though because embedding is already the cheapest stage in the pipeline, the absolute time saved by query embedding caching is modest compared to what could be saved by caching further downstream.

Caching at the candidate-pool level, keyed on the exact query text, offers a larger latency benefit precisely because it can skip both the vector search stage and the reranking stage entirely for a repeated query, but it introduces a staleness risk: a cached candidate pool computed before a relevant document was ingested, updated, or removed will continue to omit or include chunks that no longer reflect the corpus's current state until that cache entry expires or is explicitly invalidated.

Any caching layer in this pipeline must be scoped to a specific embedding model version and a specific corpus state, for the same reason raw embeddings cannot be compared across model versions; a cached result computed under a previous embedding model or against a corpus snapshot that has since changed is not a safe substitute for recomputation, even though reusing it would appear to execute successfully and return some plausible-looking result.

## Failure Modes

Cascading latency occurs when a slow stage early in the pipeline, most often the OpenRouter decomposition call, delays every subsequent stage that depends on its output, since vector search cannot begin until the set of queries to search for is known, which means a slow or retried decomposition call adds its full delay on top of, rather than in parallel with, the rest of the pipeline's normal latency.

A reranker timeout or failure requires a defined fallback behavior rather than simply failing the entire request, since a cross-encoder scoring pass over a large candidate pool is the single most time-consuming stage and therefore the stage most likely to occasionally exceed an acceptable latency bound; the documented fallback in this system is to return the vector-search-ordered candidate pool directly, unranked, rather than returning no results at all, trading ranking quality for availability in that specific failure scenario.

Partial pipeline failure describes the case where decomposition succeeds but produces subqueries that are redundant with each other or drift away from the original query's intent, which does not cause an outright error anywhere in the pipeline but can silently degrade the quality of the merged candidate pool, since redundant subqueries duplicate effort without adding new relevant evidence, while off-topic subqueries introduce irrelevant candidates into the pool that the reranker must then spend time correctly demoting.

## Operational Considerations

Autoscaling the retrieval service needs to account for the fact that reranking latency scales with candidate pool size per request, not just with request volume, so a traffic pattern shift toward queries that trigger larger effective candidate pools, for example through decomposition producing more subqueries, can increase per-request compute cost even at a constant request rate, which is a different scaling dimension than simply handling more requests per second.

Batching reranking requests across multiple concurrent user queries, when the underlying cross-encoder model and serving infrastructure support it, can improve throughput in the same way batched embedding encoding does, though reranking batches are necessarily smaller and more variable in size than embedding batches, since each query's candidate pool size can differ and all candidates for a given query must be scored before that query's result can be returned.

## Request Timeout Budgets and Partial Results

An overall request timeout budget is allocated across the full pipeline so that a single slow stage cannot cause a request to hang indefinitely, and because reranking is both the dominant and the most variable-latency stage, most of the available timeout budget is implicitly reserved for it, with embedding, vector search, and merge-and-dedup together expected to consume only a small fraction of the total budget under normal conditions.

When the decomposition stage alone consumes an unexpectedly large share of the timeout budget, for example due to a slow OpenRouter response, the remaining budget available for vector search and reranking shrinks accordingly, which can force the pipeline into a degraded mode, such as proceeding with only the original query rather than waiting for subqueries that have not yet returned, rather than exceeding the overall request timeout entirely.

A partial result, meaning a reranked result set produced under a reduced effective candidate pool because one or more subqueries did not complete within the available time budget, is still returned to the caller rather than treated as a failed request, since a reranked result based on incomplete decomposition is generally considered more useful than no result at all, which mirrors the same reasoning behind falling back to vector-search-ordered results when reranking itself fails.

## Pipeline Behavior Under Concurrent Load

Under concurrent load, the reranking stage is the first to exhibit queueing effects, since it is both the most compute-intensive stage per request and the stage whose cost scales with candidate pool size rather than being a small constant amount of work per request, which means a burst of concurrent requests with large candidate pools can create reranking queue depth even while the embedding and vector search stages remain comfortably within capacity.

Backpressure in this pipeline is applied primarily by bounding how many requests can be concurrently in the reranking stage at once, rather than by bounding concurrency earlier in the pipeline, since allowing embedding and vector search to proceed without restriction for requests that are merely waiting their turn for reranking capacity does not meaningfully worsen the bottleneck, while restricting concurrency at the reranking stage specifically prevents an unbounded number of large candidate pools from being scored simultaneously and exhausting available compute.

A sustained period of concurrent load high enough to create meaningful reranking queue depth will manifest first as an increase in p95 and p99 end-to-end latency, well before p50 latency is noticeably affected, since the typical request still completes close to its usual latency while a growing minority of requests wait behind a backlog, which is consistent with why percentile-based latency monitoring, rather than average latency alone, is relied upon for detecting this kind of load-driven degradation.

## Health Checks and Dependency Readiness

Before the retrieval service accepts live traffic, a readiness check verifies that each of its external dependencies, the PostgreSQL database holding chunk embeddings, the embedding provider, and the cross-encoder reranker deployment, is actually reachable and responding, rather than only checking that the retrieval service's own process has started successfully, since a service that reports itself as healthy while one of its dependencies is actually unreachable would otherwise begin accepting and then failing requests immediately.

The query decomposition dependency on OpenRouter is treated differently in this readiness check than the other three dependencies: because decomposition failure already has a documented graceful fallback of treating the original query as the sole subquery, an unreachable OpenRouter endpoint does not block the service from reporting itself ready, whereas an unreachable database, embedding provider, or reranker, none

of which have an equivalent fallback that still produces a meaningful result, does block readiness, since no later pipeline stage can substitute for any of those three.

This distinction reflects the same architectural asymmetry described elsewhere regarding decomposition's optional status relative to the rest of the pipeline: decomposition is a genuinely optional enhancement layered in front of a pipeline that functions correctly without it, while embedding, vector search, and reranking are each a required stage with no equivalent fallback path that still produces a meaningful ranked result if that stage is entirely unavailable.

## Limitations

This architecture has no sparse retrieval, no BM25 scoring, and no hybrid combination of sparse and dense signals anywhere in the pipeline; every candidate that reaches reranking was surfaced purely through dense vector similarity, which means a query whose correct answer depends on an exact rare term that the embedding model has no strong representation for has no lexical-match fallback available to recover that chunk if dense retrieval ranked it outside the candidate pool.

There is no automatic query-complexity classifier or router deciding whether a given query should be decomposed; the decomposition decision is made by the language model invoked at that stage for every query that reaches it, which means the system has no separate, cheaper mechanism to quickly identify queries that obviously do not need decomposition before incurring the cost of asking the decomposition model to evaluate them.

There is no parent-child or hierarchical chunk retrieval mechanism either; every chunk is retrieved, merged, and reranked as an independent, flat unit with no linkage back to sibling or parent chunks from the same document section, so the pipeline has no built-in way to expand a retrieved chunk with its immediately surrounding context if that chunk alone turns out to be insufficient for a downstream consumer.